Assignment 2

1. Introduction

You are a newly hired software engineer at **DataVault Solutions**, a company that manages massive, deeply nested digital archives for media agencies. Their primary server contains millions of files distributed across thousands of directories.

Currently, the support team is frustrated. Every time a client requests a specific file, the support staff uses the default operating system search. Because the search reads the disk in real-time, it takes several minutes to locate a file hidden deep within the subfolders. Time is money, and the clients are complaining about the delay.

Your manager has tasked you with building a custom, high-speed file search utility called **Turbo-Path**. Instead of searching the hard drive dynamically every time a query is made, Turbo-Path must pre-index the file system. Upon startup, the program needs to traverse the target directory, load all folders, subfolders, and files into an efficient in-memory data structure, and then wait for user input. When a user searches for a file by name, the system must instantly return the **absolute path** to that file.

2. Learning Objectives

By completing this project, you will:

- Evaluate and implement optimal data structures for fast lookups.
- Understand the trade-offs between memory consumption (Space Complexity) and search speed (Time Complexity).
- Design a user-friendly Command Line Interface (CLI) .

3. Requirements

Phase 1: Initialization & Indexing

- The program must accept a "root" directory path upon startup.
- It must traverse this directory and all its subdirectories.
- It must map the hierarchical structure into a custom data structure held in RAM.

Phase 2: The Search Operation

- The user should be able to type a file name (e.g., project_report.pdf or just project_report).
- The search operation must rely **only** on your data structure, not the OS disk search.
- The search time should be exceptionally fast (aiming for O(1) or O(log n) time complexity).
- The output must display the full directory path to the file (e.g., C:\DataVault\Clients\2023\Q4\project_report.pdf).
- *Bonus Handling:* If multiple files share the same name in different folders, the program should list all their paths.

Phase 3: Real-Time Updates (Extra Credit)

- Implement a mechanism to refresh the data structure (either manually via a "refresh" command or by listening to OS file system events) so that newly added files are recognized without restarting the whole application.

4. Constraints & Considerations

- **Memory:** The data structure must be memory efficient. Storing massive strings redundantly will crash the application on large drives. Consider how to store directory paths without duplicating parent folder names unnecessarily.
- **Data Structure Choice:** You must justify why you chose your specific data structures.

5. Deliverables

1. **Source Code:** Fully commented and structured code.
2. **README File:** Instructions on how to compile and run your program.
3. **Design Document (1-2 pages):** An explanation of the data structure(s) chosen and why.
    - The Big-O Time Complexity for your search operation.
    - The Big-O Space Complexity for your memory footprint.

Sample code to load a directory in your Program

```
#include <iostream>
#include <filesystem>
#include <fstream>
#include <string>

Using namespace std;

namespace fs = std::filesystem;

void loadDirectory(const fs::path& directoryPath) {
   if (!fs::is_directory(directoryPath)) {
      cout << "Error: Not a valid directory path." << endl;
      return;
   }

   cout << "Loading directory: " << directoryPath <<endl;
   for (const auto& entry : fs::directory_iterator{ directoryPath }) {
      fs::path entryPath = entry.path();
      cout << "Found entry: " << entryPath.filename() << endl;

      // Check if it is a regular file and process it
      if (fs::is_regular_file(entryPath)) {
         cout << "  -> It's a file. Opening for reading..." << endl;
         ifstream file(entryPath);
         if (file.is_open()) {
            string line;
            while (getline(file, line)) {
               // Process file content here
            }
            file.close(); // Important to close the file after use
         }
         else {
            cout << "  -> Error opening file: " << entryPath << endl;
         }
      }
      // Check if it is a subdirectory and recurse (if needed)
      else if (fs::is_directory(entryPath)) {
         cout << "  -> It's a directory." << endl;
         // Optionally, call loadDirectory(entryPath) for recursive loading
      }
```

```
    }
}

int main() {
    // Specify the path to the directory you want to load
    // "." refers to the current working directory
    // "path/to/your/directory" for a specific path
    loadDirectory(".");

    return 0;
}
```

Project Settings:

Visual Studio fully supports the <filesystem> library, but new C++ projects often default to an older C++ standard (usually C++14) for backward compatibility.

Here is exactly how to change your project settings to use C++17 so the std::filesystem error disappears.

How to Enable C++17 in Visual Studio 2022

1. **Open Project Properties:** In the **Solution Explorer** panel (usually on the right side of your screen), right-click on your **Project Name** (make sure you click the project, not the "Solution" at the very top) and select **Properties** at the bottom of the menu.
2. **Select All Configurations:** At the top left of the Properties window, change the **Configuration** dropdown to **All Configurations**. This ensures your changes apply whether you are running in Debug or Release mode.
3. **Navigate to Language Settings:** On the left-hand menu, expand **Configuration Properties**, expand **C/C++**, and click on **Language**.
4. **Change the Standard:** On the right side, find the row labeled **C++ Language Standard**. Click the dropdown menu next to it and change it to **ISO C++17 Standard (/std:c++17)**.
5. **Apply and Save:** Click **Apply** at the bottom right, then click **OK** to close the window.